

כ"ז באב תשפ"ה

21

מס' שאלון - 532

באוגוסט 2025

סמסטר 2025 ב

מס' מועד 93

20905/4

שאלון בחינת גמר

20905 - שפות תכנות

משך בחינה: 3 שעות

בשאלון זה 12 עמודים

מבנה הבחינה:

שאלון 532

קורס 20905

מועד 93

בשאלה 1 עמוד 3 בחלק של הגדרת הדקדוק מופיעה השורה הבאה :

Expression ::= proc (...) Expression

צריך לתקן שורה זו ל-

Expression ::= varproc (...) Expression

כלומר החלפת המילה proc במילה varproc

שאלה 1 (40 נקודות)

שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81.

בשאלה זו נשנה בשפת "וישגור" את האופן של הגדרת והפעלת פרוצדורה. תזכורת: בשפה זו לפרוצדורה יש פרמטר יחיד ששמו נקבע בזמן הגדרת הפרוצדורה. ובביטוי הפעלת פרוצדורה מספקים לו ערך בודד. נרצה לשנות ולעדכן את האופן שבו מוגדרת ומופעלת פרוצדורה. בנוסף לביטוי `proc-exp` להגדרת פרוצדורה הקיים בשפה, נרצה להוסיף ביטוי חדש בשם `varproc-exp`, המאפשר הגדרת סוג חדש של פרוצדורה שיש לה כמות פרמטרים משתנה (בדומה לקיים בשפת C עם `...`, ובדומה לקיים בפייתון עם `*args` למי שמכיר). לפרוצדורה החדשה ניתן יהיה לשלוח כמות משתנה של ארגומנטים, ובהגדרת הפרוצדורה מהסוג החדש לא נציין שם של פרמטר, אלא יהיה פרמטר מובנה בשפה בשם `args` שייוחדי לסוג הפרוצדורה החדשה, שיודע לייצג כמות משתנה של ארגומנטים המועברים אליו. כמו כן, נוסיף ביטוי חדש בשם `each-exp` היודע לבצע מעבר על רשימת ארגומנטים שנשלחו לפרוצדורה מהסוג החדש ולבצע עבור כל אחד מהם פעולה מסוימת.

להלן הדקדוקים של ביטוי הגדרת פרוצדורה מהסוג החדש ועדכון ביטוי הפעלת פרוצדורה, וכן דקדוק של הביטוי החדש למעבר על רשימת ארגומנטים:

Expression ::= proc (...) Expression

`varproc-exp (body)`

Expression ::= (Expression { Expression }^{})*

`call-exp (rator rands)`

Expression ::= each identifier in args do Expression

`each-exp (id exp)`

- ביטוי `varproc-exp` ביטוי של הגדרת פרוצדורה מהסוג החדש בנוסף לאופן הקיים כבר בשפה. בביטוי ההגדרה של הפרוצדורה מהסוג החדש, המתכנת מציין רק את גוף הפרוצדורה. לפרוצדורה יהיה מובנה בשפה פרמטר מיוחד בשם `args` שיודע לקבל רשימת ארגומנטים שיישלחו אליו בזמן הפעלת פרוצדורה. ביטוי `varproc-exp` מחזיר ייצוג של הפרוצדורה החדשה שהוגדרת.
- ביטוי `call-exp` בנוי מ- `rator` שהוא ביטוי המייצג פרוצדורה שרוצים להפעיל, ומ- `rands` שהם אוסף ביטויים בכמות שרירותית, שאת ערכן יש לשלוח כארגומנטים לפרוצדורה אותה רוצים להפעיל. הביטוי מחזיר כרגיל את תוצאת הפעלת הפרוצדורה. ביטוי `call-exp` משמש הן להפעלת הפרוצדורה מהסוג החדש והן להפעלת פרוצדורה מהסוג הישן הקיים בשפה.
- ביטוי `each-exp` בנוי מ- `id` שהוא שם מזהה כלשהו שקובע המתכנת, שם מזהה זה יתפקד כמשתנה ייעודי במסגרת חישוב ביטוי `each-exp`, והוא יקבל בכל פעם את הערך הבא שיש במשתנה בשם `args` (שאמור להיות קיים בסביבה מעצם הגדרת הפרוצדורה) ויבצע עבורו את הביטוי `exp`. ביטוי `each-exp` יחזיר תוצאה מסוג רשימה המכילה את תוצאות פעולותיו על ערכי `args`. שימו לב, זה ביטוי שמחזיר סוג תשובה חדשה בשפה.

המשך בעמוד הבא

דוגמה 1:

```
> (run "let p= varproc(...)
      each v in args do
        -(v,2)
      in
        (p 10 20 30)")
(list-val (list (num-val 8) (num-val 18) (num-val 28)))
>
```

בדוגמה זו מוגדרת פרוצדורה בשם p עם כמות משתנה של פרמטרים, בגוף הפרוצדורה מתבצע ביטוי each-exp העובר באמצעות משתנה בשם v ייעודי לו, על רשימת הערכים שסופקו ל- args (הפרמטר המיוחד המובנה במפרש שלנו עבור פרוצדורה), ולכל אחד מהם הוא מפחית 2. הפרוצדורה מחזירה את תוצאתו של ביטוי ה-each-exp שהיא רשימה המכילה את תוצאות הפעולות של each-exp על האוסף args. לאחר הגדרת הפרוצדורה, היא מופעלת בדוגמה זו עם רשימת ערכים מספריים. וכפי שניתן לראות, התוצאה מייצגת את ההפחתה ב-2 של כל אחד מהארגומנטים שנשלחו.

דוגמה 2:

```
> (run "let p= varproc(...)
      each v in args do
        -(v,2)
      in
        (p  )")
(list-val '())
> |
```

דוגמה זו זחה לקודמת, אך ממחישה הפעלת פרוצדורה ללא שליחת פרמטרים (שזה בסדר)

המשך בעמוד הבא

דוגמה 3:

```
> (run "let w= each q in args do (q 8)
    in
    w")
```

```
⊗ ⊗ environments.scm:45:8: apply-env: No binding for args
>
```

דוגמה זו ממחישה שימוש בביטוי `each-exp` שלא במסגרת גוף פרוצדורה, ולכן לא קיים הפרמטר המובנה `args`, מה שמשבש את פעולתו של ביטוי `each-exp` ולכן נורקה שגיאה.

דוגמה 4:

```
> (run "let p= proc(a) - (a,4)
    in
    (p 12) ")
(num-val 8)
> |
```

דוגמה זו מדגימה התנהגות של פרוצדורה רגילה כפי שקיימת בשפה המקורית, והמשך עבודה רגיל מולה.

דוגמה 5:

```
> (run "let p= proc(a) - (a,4)
    in
    (p 12 56 23) ")
⊗ ⊗ interp.scm:106:11: apply: procedure expect just one argument
> |
```

דוגמה זו ממחישה הגדרת פרוצדורה `p` רגילה (בעלת פרמטר יחיד), וניסיון להפעיל אותה כפרוצדורה עם מספר משתנה של פרמטרים. מה שכמוכן אינו מתאים ולכן התקבלה שגיאה מתאימה על כך.

המשך בעמוד הבא


```
> (run "let p= varproc(...)
      each v in args do
        - (v, 2)
      in
        (p 10 )")
(list-val (list (num-val 8)))
> |
```

בדוגמה זו, הוגדרה פרוצדורה p , כסוג פרוצדורה בעלת כמות משתנה של פרמטרים, ובקריאה אליה נשלח בדיוק ארגומנט אחד (שזה גם סוג של כמות משתנה) ולכן תהליך ההפעלה הפעיל את סוג הפרוצדורה החדשה עם הרשימה של הארגומנטים, שבמקרה זה אורכה הוא 1.

```
> (run "let p = varproc(...)
      each q in args do (q 8)
      in
        (p proc (a) -(a,1) proc (b) zero?(b))")
(list-val (list (num-val 7) (bool-val #f)))
\
```

בדוגמה זו מומחשת הגדרה של פרוצדורה בעלת מספר משתנה של פרמטרים, לכל פרמטר שהגיע הפרוצדורה מפעילה אותו על 8 (מה שאומר שהפרמטרים לפרוצדורה p צריכים להיות פרוצדורות בדוגמה זו), ואכן בהפעלת p נשלחות אליה, בכמות משתנה, 2 פרוצדורות.

שימו לב:

בשאלה זו, יש סוג חדש של פרוצדורה בנוסף ולא במקום הסוג הקיים. וכן, יש ביטוי חדש למעבר על args המחזיר סוג תשובה חדשה בשפה. וכן ביטוי הפעלת פרוצדורה ותהליך הפעלתה צריכים להתעדכן ולהיות מותאמים ל-2 סוגי הפרוצדורות שיש כעת בשפה.

ממשו והטמיעו את השינויים הדרושים כפי שהוגדרו והודגמו בתוך שפת "וישגור" (שפת PROC).

בתשובתכם, הקפידו להסביר היכן בדיוק נדרשים שינויים ותוספות בחלקי המפרש, ומהם השינויים והתוספות.

לא תתקבל תשובה של תיאור מילולי. יש לממש בקוד את השינויים הדרושים.

הנחיות כלליות לפתרון:

- (1) הקפידו על כתב ברור, ועל קוד מבני ומסודר.
- (2) הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
- (3) הפתרון אמור לשמור על הגדרות השפה המקוריות (פרט למקרים בהם נדרש שינוי כזה במפורש בשאלה)
- (4) על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
- (5) בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
- (6) ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב-terminals, non-terminals ופעולות של סגור (}) או פתוח ({)

שאלה 2 בעמוד הבא

שאלה 2 (35 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת "וירמוז" (IMPLICIT-REFS).

בשאלה זו, נרצה להוסיף לשפה עבודה עם ערכים לא מאותחלים, וקבלת אזהרה בזמן ריצה על כל שימוש בהם.

כלומר, אם משתנה מקבל להיות שווה לערך לא מאותחל, ובהמשך יש שימוש בערכו של המשתנה (למשל ביצוע פעולת חיסור איתו), נרצה שתתקבל הדפסת התראה (לא שגיאה) על כך שיש שימוש במשתנה לא מאותחל. אם בהמשך התוכנית יושם במשתנה ערך, עיי פעולת ההשמה שיש בשפה, לא יתקבלו התראות לגביו, כי הוא הפסיק להכיל ערך לא מאותחל. נרצה גם התנהגות דומה לשימוש בערך לא מאותחל שחוזר כתוצאה מחישוב ביטוי, ונעשה בו שימוש לאחר מכן, מבלי לאחסנו במשתנה.

כלומר מטרת השאלה, היא הוספה לשפה של סוג ערך חדש, המייצג ערך לא מאותחל (ערך זבל) ובכל פעם שיש ניסיון לשימוש בערך זה במסגרת ביצוע פעולות תתקבל הדפסת התראה על כך לפני ביצוע הפעולה.

נוסיף לשפה ביטוי חדש שתוצאתו היא ערך לא מאותחל. למען הפשטות נחליט שהערך הלא מאותחל יהיה באופן שרירותי המספר 900 (ערך זבל). יש להבדיל זאת מהערך 900 num-val שמייצג תוצאה תקינה. נבחר זאת בדוגמה בהמשך.

נוסיף לשפה את הדקדוק הבא של ביטוי המחזיר ערך זבל של 900:

Expression ::= ???

uninit-exp ()

ביטוי uninit-exp הוא ביטוי חדש שתוצאתו היא ערך לא מאותחל. (הוסבר קודם שהערך יהיה שרירותי 900). שימו לב יש כאן סוג תשובה חדשה בשפה. על מנת להבדילה מתשובה מספרית תקינה של 900

נסתכל כעת על 2 דוגמאות שימוש שימחישו כיצד ההתנהגות הרצויה שהוסברה בתחילת השאלה באה לידי ביטוי. ולאחר הדוגמאות נסביר ונדגיש שוב את דרישות המימוש בעבודה עם ערכים לא מאותחלים.

המשך בעמוד הבא

דוגמה 1:

```
> (run "let z=???  
  in  
    let w= 50  
    in  
      let y= proc (a) -(a,5)  
      in  
        begin  
          -(z,w);  
          (y w);  
          (y z);  
          set z = 270;  
          -(z,w);  
          (y z)  
        end")
```

Warning: you are using uninitialized variable z
Warning: you are using uninitialized variable z
Warning: you are using uninitialized variable a
(num-val 265)

>

נסביר בפירוט דוגמה זו:

בדוגמה מוגדר משתנה z לא מאותחל (הוא קיבל ערך לא מאותחל המיוצג שרירותית ע"י ערך זבל של 900), לאחר מכן מוגדר משתנה w המאותחל ל-50 (לא כערך זבל), לאחר מכן מוגדרת פרוצדורה y עם פרמטר בשם a , המחזירה $a-5$. לאחר הגדרות אלה מבוצעות מספר פעולות בבלוק `begin`.

הפעולה הראשונה, מבצעת פעולת חיסור בין z לבין w , ומאחר ו- z נחשב כלא מאותחל ניתן לראות שקיבלנו בפלט התוכנית אזהרה על שימוש במשתנה z שאינו מאותחל, פעולת החיסור מבוצעת בין 900 שיש ל- z לבין 50 שיש ל- w . הפעולה השניה בבלוק הפעולות היא הפעלת הפרוצדורה y עם הפרמטר w , שערכו נשלח ל- a , לכן a מקבל ערך מאותחל של 50, וביצוע הפרוצדורה מחזיר 45, ומאחר ולא היה כאן שימוש בערך לא מאותחל אז לא התקבלה התראה על פעולה זו.

הפעולה השלישית בבלוק הפעולות היא שוב הפעלת הפרוצדורה y , הפעם עם המשתנה הלא מאותחל z , פעולת הפעלת פרוצדורה מחשבת את ערכו של הארגומנט שיש לשלוח לפרמטר a , ומאחר ומדובר על בירור ערכו של z , מודפסת התראה על שימוש במשתנה z לא מאותחל (זו ההתראה השניה שהתוכנית הדפיסה), לאחר מכן עוברים לבצע את הפרוצדורה, שבה הפרמטר a שלה קיבל ערך לא מאותחל שנשלח אליו, ובביצוע הפרוצדורה עצמה יש ניסיון לבצע חיסור עם a , ושוב יש כאן שימוש במשתנה לא מאותחל, ולכן מתקבלת התראה שלישית על שימוש במשתנה a לא מאותחל. שימו לב שבהתראה על שימוש במשתנה לא מאותחל מודפס גם שמו של המשתנה. הפעולה הבאה בבלוק ההוראות שמבוצעת היא הפעולה הרביעית, שבה מבצעים השמה למשתנה z הלא מאותחל, ולראשונה מאתחלים אותו לערך של 270. מעתה z נחשב כמאותחל ולא אמורות להתקבל לגביו התראות בניסיון להשתמש בו. ואכן כפי שניתן לראות, הפעולות הבאות שמבוצעות בבלוק ההוראות, עושות שימוש במשתנה z , שכעת מאותחל ולכן לא התקבלו עוד התראות. התוכנית החזירה את תוצאת הפעולה האחרונה שבוצעה בבלוק ה-`begin`.


```
> (run "let p = proc (a) ???
      in
        -((p 7) , 20)")
```

Warning: you are using uninitialized value
(num-val 880)

```
> |
```

דוגמה זו ממחישה שימוש בערך לא מאותחל שלא במסגרת פניה למשתנה, אלא שימוש בערך לא מאותחל שהתקבל כתוצאה מחישוב ביטוי, ושימוש מיידי בערך זה. כאן מתקבלת התראה כללית יותר על שימוש בערך לא מאותחל.

בדוגמה, מוגדרת פרוצדורה בשם p המקבלת פרמטר a ומחזירה כתוצאה ערך לא מאותחל. בהמשך מבוצעת פעולת חיסור בין תוצאות הביטויים: הפעלת p על 7, לבין המספר 20. מאחר ויש שימוש בערך לא מאותחל שחזר מהפעלת הפרוצדורה p , התקבלה התראה על כך, ולאחר מכן פעולת החיסור בוצעה בין 900 הערך הלא מאותחל לבין 20, ולכן תוצאת התוכנית היא 880.

נסכם את דרישות השאלה:

- יש להגדיר ביטוי חדש המחזיר ערך לא מאותחל (הוחלט שהוא ייוצג ע"י ערך הזבל 900) – זה סוג תשובה חדש בשפה.
- בכל פעם שיש גישה למשתנה בשפה לקבלת ערכו, צריכה להתקבל התראה במידה וערכו לא מאותחל (ההתראה צריכה לכלול את שמו של המשתנה)
- כאשר משתמשים בערך לא מאותחל שלא דרך פניה למשתנה, אלא כערך שמתקבל מסיום חישוב ביטוי, גם יש להפיק התראה על כך, כפי שהודגם למשל בדוגמה 2.
- לצורך הדפסת התראה משתמשים ב- `eopl: printf` שיש לה שימוש שדומה ל- `eopl: error` אלא שלאחר ההדפסה ביצוע התוכנית ממשיך, בניגוד לזריקת שגיאה שממנה לא חוזרים להמשך ביצוע התוכנית.

עליכם להטמיע בשפה התנהגות ועבודה עם ערך לא מאותחל כפי שהוסבר והודגם.

רמז: אין צורך ללכת לכיוון של פתרון המשנה את כלל הביטויים בשפה, אפשר לממש זאת בדרך קצרה מאוד.

ממשו והטמיעו את השינויים הדרושים בתוך שפת "וירמוז" (שפת IMPLICIT-REFS).

בתשובתכם, הקפידו להסביר היכן בדיוק נדרשים שינויים ותוספות בקבצי המפרש, ומהם השינויים והתוספות.

הנחיות כלליות לפתרון:

- (1) הקפידו על כתב ברור, ועל קוד מבני ומסודר.
- (2) הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
- (3) הפתרון אמור לשמור על הגדרות השפה המקוריות. (כגון אופן ניהול הסביבה וסוגי הערכים בה, אופן ניהול ההחסן והערכים שניתן לשמור בו, וכדומה)
- (4) על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
- (5) בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
- (6) ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב-terminals, non-terminals ופעולות של סגור (}*) או פתוח ({+})

שאלה 3 בעמוד הבא

שאלה 3 (25 נקודות)

שימו לב – זוהי שאלה מפוצלת ל-2 גרסאות שונות. עליכם לפתור את הגרסה המתאימה לסמסטר הלימוד בו למדתם את הקורס. פתרון של גרסה שגויה לא יזכה בניקוד

גרסה עבור סטודנטים שלמדו בסמסטר 24

להלן תוכנית בשפת PROC (שפת "וישגור")

```
(proc (a) (proc (b) -(a,b) 500) 100)
```

רשמו את שלבי חישוב התוכנית ע"י המפרש תחת סביבה התחלתית ריקה, באמצעות שימוש בסימונים מוסכמים כמתואר למשל בספר הלימוד בעמודים 77-78.

לבסוף רשמו מהי תוצאת התוכנית?

גרסה עבור יתר הנבחנים

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

```
let p = proc (a:?) if a then a else zero?(70)
in
  let q = proc (b:?) (p b)
  in
    -((q 40) , 20)
```

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים). לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

א. (10 נקודות)

הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.

ב. (15 נקודות)

פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש באלגוריתם שנלמד בפרק 7 בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו האם התוכנית מוטפסת היטב? ואם כן, מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.

בהצלחה!